



Project Report

Ticket Booking System Implementation

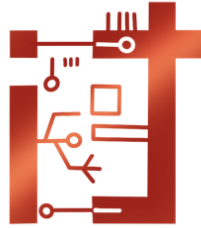
Abstract

This project implements a **BookingManager** using Dependency Injection to decouple core logic from external services like payment and notifications. By leveraging **TDD and mocking** frameworks, we ensure all User Stories are verified through isolated, robust unit tests.

Link Of the GitHub Repo

	Student Number	Section
Fatima Eid	2239352	1
Areej Al-Masri	2334600	1
Liyan Ewies	2336703	1

Instructor: Dr. Bashar Shboul



Project Report

Ticket Booking System Implementation

Table of Contents

Engineering Design: Dependency Injection Analysis 3

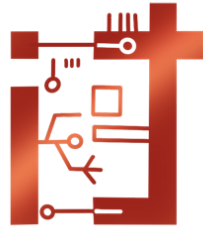
- Explanation of how DI improves system testability compared to hard-coded dependencies.

Engineering Practice: Limitations of Mocking 4

- Technical risks and limitations of relying heavily on mocking frameworks.

Individual and Team Work: Contribution Matrix 5

- Detailed breakdown of tasks, code, and documentation authored by each team member



Engineering Design: Dependency Injection Analysis

The BookingManager's use of **constructor-based Dependency Injection (DI)** is central to its testable design, as it removes the responsibility of object instantiation from the class itself. By injecting the **IPaymentGateway**, **INotificationService**, and **IEventRepository** through the constructor, the system remains decoupled from concrete, low-level implementations. In a traditional hard-coded architecture, testing would require live database connections or network access to third-party APIs, making unit tests slow and brittle. With DI, we can substitute these heavy dependencies with lightweight mock objects, ensuring that our tests focus exclusively on the internal logic of the **BookingManager** in a completely isolated and controlled environment.

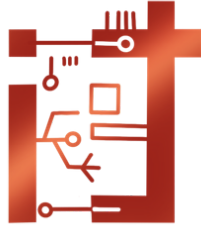
This design is particularly advantageous for **Test-Driven Development (TDD)** because it facilitates the effortless simulation of diverse scenarios, including edge cases and failure states. For example, to verify the "Sold Out" path (US-03), we can easily configure a mock **IEventRepository** to return true for **isSoldOut()** without needing to manually populate a real database. Furthermore, DI allows for precise interaction verification—using tools like Mockito to ensure that **saveBooking()** or **sendConfirmation()** are called exactly once in the happy path (US-01) or never called when input is invalid (US-02). This results in a suite of tests that are not only fast and deterministic but also highly descriptive of the system's intended behavior.



Engineering Practice: Limitations of Mocking

Although **mocking** is a great way to isolate our **BookingManager** code for testing, it isn't perfect and comes with some technical risks. The biggest issue is behavioral divergence, which basically means the mock starts acting differently than the real system. Since a mock is just a "fake" version we programmed ourselves, it only does exactly what we tell it to do. If the real **IPaymentGateway** API gets updated—for example, if it starts throwing an exception instead of returning a null value—but we forget to update our mock to match, our tests will still pass even though the real app would crash in production. This gives us a "false green" light that can be pretty misleading.

Another **limitation** is that relying too much on mocks can make our tests fragile. Sometimes we end up testing how the code is written (the implementation) instead of what it actually achieves. If we refactor the internal code but the final result stays the same, the mocks might still break, which adds a lot of extra work to keep them updated. At the end of the day, mocks are excellent for checking logic in isolation, but they can't catch bugs that happen when different parts of the system actually talk to each other. That's why we still need integration tests to make sure the real database and services work together correctly.



Individual and Team Work: Contribution Matrix

Student Name	Core Contributions
Fatima Eid	Authored the core logic for BookingManager.java and implemented input validation for US-01 and US-02
Areej Al-Masri	Developed unit tests in BookingManagerTest.java and utilized Mockito for interaction verification for US-03.
Liyan Ewies	Authored the ProjectReport.pdf and provided collaborative support for the implementation of BookingManager.java and BookingManagerTest.java